

DESAFÍO TÉCNICO · PREGUNTAS 3 A 5

Del laboratorio a producción

Cómo lograr que el modelo rinda en producción lo que rindió en el laboratorio, qué revisar si no lo hace y cómo ponerlo en marcha.

Complemento del informe de la solución · modelo de prevención de fraude orientado a ganancia

Juanfe Martínez

DATA SCIENCE · MACHINE LEARNING · PREVENCIÓN DE FRAUDE

Dataset · 150.000 transacciones

Código · github.com/JuanFeDS/proyecto

Que el laboratorio se parezca a producción

La evaluación debe reproducir las condiciones en las que el modelo va a decidir: los mismos datos disponibles, el mismo código y la misma métrica de negocio.

1 · Validar como se va a usar el modelo

1 Variables con estado aprendidas solo en entrenamiento

Tasa y frecuencia de `j` y países frecuentes se aprenden en entrenamiento y se aplican tal cual a los nuevos; la tasa de `j`, out-of-fold. Así validado, el pipeline reproduce el laboratorio: **78,9% ± 1,3** de la ganancia ideal.

2 Evaluar en el tiempo, no solo con folds aleatorios

Con folds aleatorios, una tasa de `j` puede usar etiquetas posteriores. Entrenando solo con el pasado, el modelo final da 75,3%, 73,7%, 77,1% y 79,5% en las cuatro últimas semanas (-1,2 frente a la CV aleatoria en las mismas filas; -0,4 sin la primera, entrenada con solo 60.000 transacciones); con ajuste y umbral elegidos solo con el pasado, 80,0% en la última.

3 No evaluar con los datos que se usaron para elegir

El tuning parecía aportar +0,7 puntos con los folds de la búsqueda y +0,4 con otra partición, que luego decidió también pesos, variables y umbral: reusada, deja de ser nueva. Hay que elegir con datos anteriores a los de evaluación.

4 Etiquetas maduras

Un contracargo puede llegar semanas después: con transacciones recientes, muchos fraudes todavía figuran como legítimos y la evaluación sale optimista. Hay que fijar una **ventana de maduración** (por ejemplo, 60 a 90 días) y usar solo transacciones que la cumplieron.

5 Misma frescura de las variables

En producción, la tasa de `j` solo puede usar etiquetas maduras: la validación debe simular ese desfase y la tabla refrescarse con una cadencia definida.

2 · Medir lo que importa al negocio

- **Ganancia, no accuracy:** aprobar todo acierta el 95% y obtiene solo el 63,4% de la ganancia ideal.
- **Umbral derivado del costo:** aprobar conviene si $p < 20\%$, el óptimo con CV aleatoria. En el tiempo, 15% gana ~1 punto en cada semana, pero elegido solo con semanas previas (0,12) no se distingue de 20% en la siguiente. Si cambia la tasa de fraude, se corrigen las probabilidades, no el umbral: con π la tasa de entrenamiento y π' la vigente, $p' = pa / (pa + (1-p)b)$, con $a = \pi' / \pi$ y $b = (1-\pi') / (1-\pi)$; o se recalibran con etiquetas maduras.
- **Con su incertidumbre:** ±1,3 puntos entre folds aleatorios, pero ~2,5 entre semanas (73,7% a 79,5%). Las alertas deben usar la variación semanal: una caída menor no es necesariamente degradación.

3 · El mismo código y el mismo entorno

- Un único pipeline valida, puntúa en batch y responde en la API, con tests de que una transacción recibe la misma probabilidad sola, en lote o por la API.
- Versiones de librerías desde un único lock: la imagen de la API no traía `pyarrow`, necesario para cargar el modelo, porque sus dependencias se mantenían a mano.

4 · Verificar los supuestos sobre los datos

- **Disponibilidad al decidir:** confirmar que todas las variables, en especial `score`, existen antes de aprobar. Es la más valiosa (desordenarla cuesta 8,3 puntos); sin ella, el baseline pierde 1,3.
- **Sesgo de selección:** si el dataset excluye lo que el sistema actual ya rechazó, el modelo aprendió sobre una población filtrada y, al reemplazar ese filtro, verá casos nuevos. Hay que confirmarlo y medir en sombra sobre todo el tráfico.
- **Representatividad:** 5% exacto de fraude y 45 días de pandemia; comparar con la tasa real.

Si producción rinde distinto: las causas más probables

Ordenadas de más a menos probable para una diferencia **grande**, con la señal que permite distinguir cada una.

#	CAUSA	POR QUÉ PASA	CÓMO SE DETECTA
1	Variables no disponibles a tiempo (leakage)	Si <code>score</code> u otra variable incorpora información posterior a la compra, el modelo aprendió de algo que en producción no existe. El análisis no encontró señales, pero no se puede descartar sin saber cómo se calcula.	Caída brusca desde el primer día; la distribución de la variable en producción difiere de la de entrenamiento.
2	Población distinta (sesgo de selección)	Si el dataset excluye lo que el sistema actual rechaza, el modelo nunca vio ese tráfico y sus probabilidades ahí no están calibradas.	Distribución de probabilidades y tasa de rechazo muy distintas a las del laboratorio.
3	Diferencias entre entrenar y servir	Tipos, nulos, categorías nuevas o agregados con otra ventana. En este proyecto aparecieron dos casos, ya resueltos: numéricas nulas que llegaban como texto con una sola fila, y categorías no vistas que se volvían nulo sin aviso. Y la zona horaria: la API exige la fecha con zona y la convierte a la del dataset (UTC-3, supuesta), para que la hora del día signifique lo mismo que al entrenar.	La misma transacción recibe distinta probabilidad en batch y en la API.
4	Otra tasa de fraude o de montos	El 5% del dataset probablemente no es el real, y la tasa diaria osciló entre 2,2% y 6,6%. Las probabilidades quedan desplazadas: la ganancia cae aunque el modelo ordene bien.	El AUC se mantiene y la ganancia cae; corregir las probabilidades por la tasa vigente la recupera.
5	El fraude se adapta al modelo	Los defraudadores prueban y aprenden qué pasa el control; además, el modelo cambia la población que ve (lo que rechaza no genera etiquetas). El entrenamiento corresponde a un período atípico.	Degradación gradual y concentrada en ciertos segmentos (categorías de <code>j</code> , países, montos).
6	Cambios en sistemas de los que depende	Si el sistema que genera <code>score</code> se recalibra o se reemplaza, la variable más importante cambia de significado sin que cambie el código del modelo.	Salto en la distribución de <code>score</code> sin cambios en el modelo.
7	Fallas operativas	Timeouts o caídas resueltos con una política de respaldo (por ejemplo, aprobar todo) se ven como "peor performance" aunque el modelo no haya decidido.	Proporción de decisiones tomadas por el respaldo; latencia p99.
8	Medición incompleta	Etiquetas que todavía no maduraron, transacciones rechazadas sin etiqueta y ventanas cortas (la ganancia varía ~2,5 puntos entre semanas).	La ganancia medida mejora a medida que maduran las etiquetas del mismo período.

Poner el modelo en producción

Los tres primeros pasos están implementados en el proyecto; los demás son los que faltan para operar con tráfico real.

- 1 Empaquetar como pipeline reproducible** HECHO
 Variables, modelo y umbral en un único artefacto (`FraudPipeline`), con 89 tests unitarios y de integración que corren en CI en cada push.
- 2 Validar, registrar y versionar** HECHO
 Todo queda en MLflow y el rollback es mover el alias `champion`. Un **gate de promoción** exige una validación del mismo modelo y datos y no más de 0,5 puntos bajo el champion; falta comparar a ambos sobre el mismo holdout reciente y exigir el mismo commit.
- 3 Servir** HECHO
 API en Cloud Run con validación del input (fecha con zona horaria) y API key que falla cerrada; la imagen incluye el modelo y `/salud` informa la versión. También scoring batch.
- 4 Definir latencia y respaldo** SIGUIENTE
 Medido en Cloud Run (1 vCPU, 300 requests): el servidor decide en **20 ms p50 y 28 ms p99**; fuera de la región, la red suma ~80 ms. Falta fijar el SLA del checkout, probar con carga, mantener una instancia caliente (arranque en frío: 3,8 s) y definir el respaldo (recuadro).
- 5 Desplegar gradualmente** SIGUIENTE
 Modo sombra primero: el modelo puntúa el tráfico real sin decidir. Luego champion/challenger con criterios de éxito definidos de antemano (ganancia, tasa de rechazo, fraudes detectados) y comparación pareada (recuadro).
- 6 Una tercera opción entre aprobar y rechazar** SIGUIENTE
 La política rechaza el 3% de las legítimas (4.612 compras, 93 mil de margen), y la matriz de costos no mide la fricción: abandono, reclamos y clientes que no vuelven. En la franja donde la curva de ganancia es plana conviene **revisar o pedir autenticación (3DS)** en lugar de rechazar.
- 7 Monitorear** SIGUIENTE
 Registrar cada decisión (probabilidad, decisión, versión). Sin esperar etiquetas: tasa de rechazo, distribución de probabilidades, drift de `score`, `monto`, `o` y país, y proporción de categorías de `j` no vistas. Con etiquetas maduras: fraude detectado y ganancia realizada frente a la esperada. Alertas por rango.
- 8 Reentrenar y automatizar** SIGUIENTE
 Con etiquetas maduras, cadencia fija o ante drift, recalibrando las probabilidades. Automatizar el despliegue al promover una versión (el CI ya existe) y SHAP por decisión para reclamos.

SI EL MODELO SE CAE: ¿APROBAR O RECHAZAR TODO?

Rechazar todo deja el 0% de la ganancia; aprobar todo, el 63,4%, pero sin ningún freno. El mejor respaldo es el sistema o la regla vigente (la regla por score deja el 67,4%); si no existe, aprobar con un límite de monto, porque el 20% más caro concentra el 82% de la pérdida por fraude. La verificación de la API key, en cambio, debe fallar cerrada: si falta su configuración, la API no arranca.

CHAMPION/CHALLENGER: EL LÍMITE ES LA MADURACIÓN, NO EL VOLUMEN

Con grupos independientes, detectar 1 punto (95% de confianza, 80% de potencia) requiere ~800.000 transacciones por grupo: a miles por segundo, son minutos. El cuello de botella es la maduración de los contracargos (60 a 90 días) y cubrir varias semanas, porque la ganancia varía ~2,5 puntos entre ellas. Conviene puntuar las mismas transacciones con ambos modelos (en sombra) y comparar semana a semana, dejando una pequeña fracción aprobada al azar para medir sin el sesgo de las rechazadas.